MODULE 18 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Deploying Pipelines

Source control, Cloud Build, Terraform and Infrastructure Manager, templates as artefacts, DAGs, Dataform, and testing.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 18 of 45

admin@sthithapragna.com

If it is not in Git, it does not exist. Every environment is built from the same commit.

From commit to production

01
Commit
A change is pushed and reviewed in Git.

02
Test
Unit tests on the Direct Runner, DAG checks, Dataform compile.

03
Build once
A Flex template image, tagged with the commit.

04
Preproduction
Integration and end-to-end tests, on real services.

05
Promote
The same artefact goes to production.

THE RULE

Build once, promote the same artefact. Rebuilding for production means production runs untested code.

Everything has a home

WHAT	WHERE IT LIVES	HOW IT SHIPS
Pipeline code	Git	Cloud Build, into a Flex template
Infrastructure and IAM	Terraform in Git	Infrastructure Manager
Dataflow jobs	Artifact Registry and Cloud Storage	flex-template run
Airflow DAGs	Git	Copied to the environment's dags folder
SQL pipelines	Dataform repository in Git	Release and workflow configurations
Secrets	Secret Manager	Read at run time, never committed

THE TRAP

A key or password in Git is a security incident, even in a private repository.

Where the code lives

- **GitHub, GitLab or Bitbucket**, with branches, reviews and protected main branches.
- **Cloud Source Repositories** closed to new customers on **17 June 2024**.
- Google's hosted option now is **Secure Source Manager**, regional and single tenant.
- Dataform and Cloud Build both connect to these Git hosts directly.

> CURRENCY CHECK
>
> Older questions answer 'store code on Google Cloud' with Cloud Source Repositories. Recognise it, but know it is closed to newcomers.

Cloud Build: the conveyor belt

- A **build config** in YAML lists steps. Each step runs in its **own container**.
- **Triggers** start builds on a push or a pull request.
- **Private pools** reach private networks. Artefacts go to **Artifact Registry**.
- Run each trigger as a **dedicated service account**, with only the roles it needs.

> WORTH KNOWING
>
> Cloud Build meets SLSA level 3 for supply-chain security, which matters for regulated firms.

Testing and packaging the readings pipeline

```
substitutions:
  _REPO: europe-west2-docker.pkg.dev/lk-ci-prod/pipelines
  _SPECS: gs://lk-artifacts-prod/templates
steps:
- id: unit-tests
  name: python:3.11
  entrypoint: bash
  args: ["-c", "pip install -r requirements.txt && python -m pytest tests/"]
- id: build-template
  name: gcr.io/google.com/cloudsdktool/cloud-sdk
  args: ["gcloud", "dataflow", "flex-template", "build",
         "${_SPECS}/readings-${SHORT_SHA}.json",
         "--image-gcr-path=${_REPO}/readings:${SHORT_SHA}",
         "--sdk-language=PYTHON", "--flex-template-base-image=PYTHON3"]
```

8 Unit tests first. A failure stops the build.

12-13 Image and spec are tagged with the commit, so every run is traceable.

THE PATTERN

The template built here is the one preproduction tests and production later runs.

Flex templates as release artefacts

- An **image** in Artifact Registry, plus a **spec file** in Cloud Storage.
- Launch with **flex-template run**, passing parameters for each environment.
- Same template in every project. Only the **parameters** differ.
- The launching identity needs **Service Account User** on the worker account.

> THE RULE
>
> Environment differences belong in parameters and Terraform variables, never in a separate build.

Terraform and Infrastructure Manager

- **Terraform** declares projects, datasets, topics, buckets and IAM, in Git.
- **Infrastructure Manager** runs Terraform for you, as Cloud Build jobs.
- It keeps **state** and **revisions**, and shows a **preview** before applying.
- It deploys **infrastructure**, not applications. Pipelines still ship as templates.

> THE SIGNAL
>
> 'Repeatable environments, reviewed changes, see the impact first' points to Terraform with previews.

Deploying DAGs to Managed Airflow

- DAGs are Python files in the environment's bucket, in the **dags** folder.
- Changes reach Airflow in **a few minutes**. Running tasks finish on the old file.
- Deleting a file does **not** remove the DAG's history from the Airflow UI.
- Keep **separate test and production environments**, and test DAGs in CI first.

> THE TRAP
>
> Never edit DAGs directly in the production bucket. One import error can break scheduling.

Deploying SQL pipelines

- **Release configurations** per environment: production compiles from main.
- **Schema suffixes** and **table prefixes** keep development output away from production.
- **Workflow configurations** schedule each environment's runs.
- Compile in CI, so a broken reference fails before anything runs.

> THE POINT
>
> A developer's run must never overwrite curated.readings_hh. Suffixes make that impossible.

Four levels of pipeline test

LEVEL	RUNS ON	PROVES
Unit	Direct Runner, tiny data	Each DoFn and transform is correct
Transform integration	Direct Runner, Create or TestStream	Transforms work together, including late data
System integration	Real services or emulators	Connections, schemas and permissions
End to end	Dataflow, 1% of data, then full	Scale, cost and production-like behaviour

THE TIP

For streaming, test with a second subscription on the same topic, beside production.

Separate projects, same code

SEPARATE PROJECTS

One per environment

- A sandbox per developer
- Preproduction close to production
- Production locked down
- Quotas and IAM kept apart

SAME EVERYTHING ELSE

One Terraform, one artefact

- Same modules, different variables
- Same Flex template, different parameters
- Same Dataform code, different suffixes
- Promote; never rebuild

FOR LARKSPUR

Larkspur's lk-ingest-dev, lk-ingest-preprod and lk-ingest-prod differ only in variables.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Same artefact in every environment	Build one Flex template, promote it
Infrastructure reviewed and repeatable	Terraform with Infrastructure Manager
Test a stream on **real traffic**, safely	A second subscription, in preproduction
DAGs **broke scheduling**	Git, CI tests, automated copy to the bucket
Dev runs **overwrote** production tables	Dataform release configs with suffixes
Store code on Google Cloud, new team	Secure Source Manager, or an external Git host

THE PATTERN

Nothing by hand, everything from Git, one build promoted through every environment.

> Larkspur's team builds the readings pipeline from source in the test project, and again from source when it deploys to production.
>
> Last month the production job behaved differently from the one that passed testing, because a dependency had changed in between.

What should the team change?

A Rebuild for production from the main branch, after every test passes

B Build one Flex template per commit, and promote that same artefact

C Let a senior engineer launch production jobs from their own laptop

D Switch to a classic template, rebuilt separately for each project

◆ PAUSE AND DECIDE

What should the team change?

A **Rebuild from main**

Still a second build, so a dependency can still change between them.

B **One template, promoted**

Production runs exactly the image that preproduction tested.

C **Launch from a laptop**

Unreviewed, unrepeatable, and tied to one person's machine.

D **Classic, rebuilt per project**

Two builds again, and Google recommends Flex for new work.

THE SIGNAL

Tested and production builds differ: build once, and promote the artefact.

> Larkspur wants to test a new version of the streaming readings pipeline against real production traffic for a week.
>
> The production pipeline must not lose or delay any messages during the test.

How should the test be run?

A Add a second subscription, and run the new version in preproduction

B Point the new version at the production subscription, beside the old one

C Rely on Direct Runner unit tests, using a sample of production messages

D Run the new version on production, then seek the subscription back

◆ PAUSE AND DECIDE

How should the test be run?

A A second subscription

Each subscription gets every message, so production is untouched.

B Share the subscription

Two pipelines would split the messages, and production misses half.

C Unit tests only

Useful, but they never meet real volume, timing or late data.

D Seek back afterwards

Production runs untested code first, and replay creates duplicates.

THE SIGNAL

Real traffic without touching production: a second subscription on the same topic.

> Larkspur's datasets, Pub/Sub topics and IAM bindings were all created by hand in the console. Development and production have quietly drifted apart.
>
> The security team now wants every change reviewed, and wants to see a change's impact before it is applied.

What should the platform team adopt?

A. A wiki page listing the console steps for building each environment

B. Shell scripts of gcloud commands, run by whoever needs a change to be made

C. A fresh copy of the production project, taken each time dev is needed

D. Terraform in Git, applied through Infrastructure Manager with previews

◆ PAUSE AND DECIDE

What should the platform team adopt?

A A wiki of steps

Still manual, so environments keep drifting.

B gcloud scripts

No state, no preview, and no record of what is deployed.

C Copy production

Copies production data and access into a looser environment.

D Terraform and Infra Manager

Reviewed in Git, previewed first, with state and revisions kept.

THE SIGNAL

Drift, reviews and previews: infrastructure as code, applied by a managed service.

> Larkspur's engineers edit Airflow DAGs directly in the production environment's bucket. Last week a DAG with an import error disrupted scheduling for an hour.
>
> The team wants reviews and automatic checks before any DAG reaches production.

What should the team do?

A Restrict bucket edits to two senior engineers, who check every change by eye

B Leave DAGs in the bucket, and watch the Airflow UI for import errors

C Keep DAGs in Git, test them in CI, and let Cloud Build copy them across

D Write DAGs straight into the Airflow metadata database instead

◆ PAUSE AND DECIDE

What should the team do?

A Two careful engineers

Fewer people, same risk, and still no review or tests.

B Watch for errors

Finds the break after it has happened, not before.

C Git, CI, Cloud Build

Reviewed, tested for import errors, then copied by automation.

D The metadata database

DAGs are files in the bucket. That database is not for code.

THE SIGNAL

Broken DAGs in production plus a wish for reviews: Git, CI checks, automated deployment.

Module 18 in five lines

- Everything lives in **Git**. Secrets live in **Secret Manager**.
- **Build once** with Cloud Build, then promote the same Flex template.
- **Terraform** through **Infrastructure Manager**, with previews and state.
- DAGs tested in CI, then copied to the bucket. Dataform suffixes protect production.
- Cloud Source Repositories is **closed to new customers**. Test streams on a second subscription.

COMING UP

Next: section 3 begins, with module 19, choosing a storage service.